

HyperNPA: Amortized Image-Conditioned Neural Particle Automata with Spatial Token Adapters

mosure
mitchell@mosure.me
Independent Research
Remote, USA

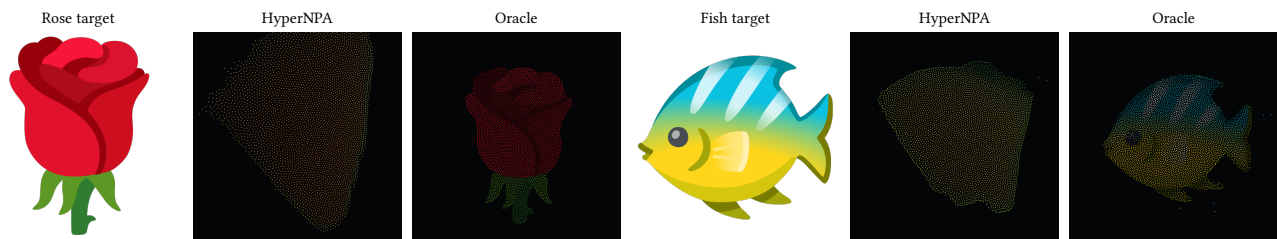


Figure 1: Same-target, same-seed, 4,096-particle rollouts at step 1,024. HyperNPA receives each target only as a held-out condition image; the oracle is the released independently trained target-specific NPA. All particle images are unmodified captures from the Bevy/WGPU Gaussian-splatting renderer. HyperNPA recovers coarse color and occupancy but not the oracle’s morphology.

Abstract

Neural Particle Automata (NPA) encode a morphology in a compact local update rule, but existing morphogenesis results train one rule per target. We study *HyperNPA*: feed-forward generation of a target-specific NPA controller from a single RGBA image. A frozen DINOv2 ViT-S/14 preserves its class token and 16×16 patch grid; a module-aware cross-attention hypernetwork maps these 257 spatial tokens to identifiable full-rank deltas for a shared 10.9k-parameter NPA; and gradients from an alpha-aware differentiable particle rollout train the shared rule and generator end-to-end. We implement the full path in Rust with Burn and device-side tiled perception and splatting adjoints.

We introduce a leakage-safe evaluation protocol that separates released target-specific NPA oracles, non-amortized per-identity adapter controls, and identity-disjoint image-conditioned inference. On a deterministic OmniSVG-1k split, the selected model obtains 11.63 dB aggregate composited PSNR and 9.32 dB p10 at 2,048 particles and 1,024 steps; correct conditioning adds 2.79 dB over shuffled images and generated deltas add 3.26 dB over the shared base. A historical 16-target direct-adapter run reached 23.58 dB, but used a per-image output bias absent from the released Growing NPA and is excluded from parity claims. On two strictly held-out targets with released NPA checkpoints, HyperNPA obtains 9.63–11.65 dB versus 27.45–27.93 dB for target-specific oracles. Dense 4,096-particle renderer captures expose failures that white-background PSNR can obscure, including coarse silhouettes, color-only matches, and collapsed particle masses. These results establish an auditable end-to-end baseline and conditional controllability, but not oracle-quality generalization. The maintained trainer now includes a conditional row-flow generator; no row-flow quality is retroactively attributed to the deterministic checkpoint evaluated here.

Keywords

neural particle automata, hypernetworks, self-organization, particle systems, morphogenesis, amortized adaptation

1 Introduction

Neural Cellular Automata learn a shared local rule whose repeated application produces global structure [7]. Neural Particle Automata (NPA) move this principle from a static grid to a Lagrangian particle system: particles carry continuous positions and latent states, perceive local neighbors through differentiable smoothed-particle-hydrodynamics (SPH) operators, and repeatedly apply a small shared MLP [5]. The released NPA models form detailed, persistent morphologies with only about 11k learned parameters. The cost is specialization: each target image is optimized into a separate dynamical rule.

This work asks whether that optimization can be amortized. Given an unseen image c , can one feed-forward network emit a controller that makes a shared particle substrate self-organize into c ? This is more demanding than direct image synthesis. The generated weights must induce useful local communication, transport thousands of particles from an unstructured seed, assign color, and remain stable under hundreds of recurrent updates.

Hypernetworks provide a natural mechanism for condition-dependent weights [2], while LoRA-style factorizations provide a convenient adapter contract [3]. However, three ambiguities make an apparently strong result easy to misinterpret. First, a per-identity adapter table is a capacity control, not an image-conditioned model. Second, regressing non-identifiable LoRA factors can reward coordinate agreement without functional agreement. Third, whole-image PSNR on transparent assets can be dominated by empty background. We therefore build the study around functional rollout loss, canonical weight deltas, identity-disjoint splits, condition-shuffle and shared-base controls, tail metrics, long horizons, and actual renderer captures.

Our contributions are:

- an end-to-end image-to-NPA architecture that preserves all DINOv2 spatial tokens and generates module-structured controller deltas;
- a Burn-native GPU path for independent per-image roll-outs, with resident data and tiled perception and splatting adjoints;
- a strict evaluation hierarchy separating released per-target oracles, direct-adapter memorization controls, and held-out amortized inference; and
- a reproducible negative result: conditional control is active and temporally stable, but broad HyperNPA quality remains far below independent NPA oracles, with evidence locating the dominant gap in the shared recurrent substrate and its training objective.

Revision and artifact boundary. This manuscript was refreshed on July 26, 2026. Its numerical quality claims remain attached to the serialized module_token_cross_attention_lora_v2 checkpoint. The maintained source tree additionally implements a versioned conditional row-flow architecture with noisy controller sources, a timestep-conditioned velocity transformer, and a deterministic Heun solve. That path has passed bounded controllability and throughput checks, but no identity-disjoint 1k/10k quality run has established oracle parity. We therefore describe it only as post-study implementation status in Appendix C, not as a relabeling of the measured model.

2 Background and Related Work

Neural automata. Growing NCA demonstrated that differentiable local rules can grow and repair images from a seed [7]. Isotropic variants remove grid orientation cues [6]. NPA instead represents each cell as a moving particle and replaces grid convolution with compact SPH perception [5]. This concentrates computation where particles exist and naturally supports varying discretizations. Our NPA state, update rule, stochastic mask, seed, and SPH feature contract follow the released growing-2D formulation; our contribution is conditional controller generation and its evaluation.

Hypernetworks and adapters. Hypernetworks generate parameters for another network and can be optimized through the downstream task [2]. LoRA writes an update as $\Delta W = (\alpha/r)BA$ [3]. Raw factors are non-identifiable: $BA = (BR)(R^{-1}A)$ for any invertible R . We use the LoRA artifact and runtime contract, but fix one factor to an identity embedding at full rank. The conditioner therefore predicts the identifiable entries of ΔW and bias deltas directly. This is a full-capacity controller test, not a parameter-efficient low-rank claim.

Visual conditions. DINOv2 produces general-purpose visual features with a ViT patch representation [9]. At 224^2 resolution, ViT-S/14 emits one class token and 256 patch tokens of width 384. Rather than average these tokens, HyperNPA preserves the grid so adapter modules can attend to layout. We use rasterized illustrations from MMSVG-Illustration, a subset associated with the MMSVG-2M collection introduced by OmniSVG [11].

3 Method

3.1 Growing Neural Particle Automaton

Each particle i has position $x_i^t \in \mathbb{R}^2$ and state $s_i^t \in \mathbb{R}^{16}$. Within support radius $\epsilon = 0.1$, SPH operators form the perception vector

$$z_i^t = [s_i^t, \bar{s}_i^t, \nabla s_i^t, \nabla \rho_i^t] \in \mathbb{R}^{66}, \quad (1)$$

where vector gradients receive the logarithmic normalization used by NPA [5]. A shared two-layer MLP predicts position and state increments:

$$u_i^t = W_2(c) \text{ReLU}(W_1(c)z_i^t + b_1(c)) + b_2(c), \quad (2)$$

$$\Delta x_i^t = \alpha \epsilon \frac{u_{i,1:2}^t}{1 + \|u_{i,1:2}^t\|_2}, \quad \Delta s_i^t = u_{i,3:18}^t, \quad (3)$$

with width 128 and $\alpha = 0.5$. Positions are detached inside perception, as in the released dynamic-particle recipe, while state gradients remain active. A Bernoulli mask $m_i^t \sim \text{Bernoulli}(0.5)$ applies asynchronous Euler updates. The base controller contains $128 \cdot 66 + 128 + 18 \cdot 128 + 18 = 10,898$ parameters.

3.2 Image Condition Tokens

The RGBA condition is resized to 224×224 . Frozen DINOv2 ViT-S/14 runs without an autograd graph and returns 257×384 features. We append patch-aligned RGB scaled by four and alpha scaled by one, obtaining

$$Z_c \in \mathbb{R}^{257 \times 388}. \quad (4)$$

The class token receives image-level RGB/alpha summaries; the remaining tokens retain the 16×16 patch layout. During the 1k experiment, all token tensors occupy 0.371 GiB and remain device resident. DINO executes in bounded batches of 64 during staging and is never differentiated.

3.3 Module-Structured Adapter Generator

For each adapter chunk q , a learned module/rank query attends to every token:

$$e_j = \text{ReLU}(Z_{c,j}W_e + b_e), \quad (5)$$

$$a_{qj} = \text{softmax}_j(d^{-1/2}q_q^\top e_j), \quad (6)$$

$$h_q = \text{ReLU}\left(q_q + \sum_j a_{qj}e_j\right), \quad (7)$$

$$v_q = W_o h_q + b_{o,q}. \quad (8)$$

The evaluated 1k checkpoint uses architecture v2: one dot-product attention map per 256-coordinate chunk, hidden width 256, and one deterministic generation step. It has 316,928 trainable parameters. The current v3 code partitions the hidden representation into four attention heads; Section 5.5 benchmarks that implementation, but we do not substitute its training loss for the held-out quality result.

The rank-82 artifact layout contains 28,026 factor coordinates. In canonical mode one factor for each matrix is a fixed identity embedding, only the coordinates needed for the dense matrix delta are trainable, and both bias deltas are generated. Thus the effective conditional output is exactly 10,898 identifiable values:

$$W_\ell(c) = W_\ell^{(0)} + \Delta W_\ell(c), \quad b_\ell(c) = b_\ell^{(0)} + \Delta b_\ell(c). \quad (9)$$

The terminology “LoRA” describes serialization and application; the tested rank is deliberately full-capacity. Despite historical artifact names, this one-step deterministic decoder is *not* rectified flow. A flow model would require noisy adapter states, timestep-conditioned velocity training, and multi-step sampling, none of which is used in the reported model.

3.4 End-to-End Rollout Training

Each optimizer batch contains eight independent condition images and eight independent particle trajectories. The selected refinement uses 1,024 particles, samples $T \sim [32, 96]$, applies full backpropagation through time, and maintains eight persistent pool states per training identity. Every four optimizer steps, two pool slots per selected identity are reset to the uniform-circle seed; live-particle-centered brush damage has radius 0.1.

Particles are centered to the target and splatted onto a 128^2 grid with unit-pixel Gaussian width. The last three state channels, shifted by 0.5, are particle colors. Let C, D and C^*, D^* denote pre-multiplied color and density. The training objective is

$$\begin{aligned} \mathcal{L}_{\text{image}} &= 5\mathcal{L}_{\text{color}} + \mathcal{L}_{\text{density}} + 10\mathcal{L}_{\text{comp}}, \\ \mathcal{L} &= 2\mathcal{L}_{\text{image}} + 0.01\mathcal{L}_{\text{disp}} \\ &\quad + 100\mathcal{L}_{\text{bound}} + 100\mathcal{L}_{\text{overflow}}. \end{aligned} \quad (10)$$

$\mathcal{L}_{\text{color}}$ and $\mathcal{L}_{\text{density}}$ use the released $\ell_1 + \ell_2$ form; color is multiplied by a detached density-error gate so shape is learned before appearance. $\mathcal{L}_{\text{comp}}$ is squared error after alpha compositing over white. No adapter teacher loss is used. Adam uses generator and base learning rates 5×10^{-5} and 5×10^{-6} , cosine decay to 0.25 $\times$, per-parameter gradient normalization, and unit-norm clipping. The 2,500-step refinement is warm started from the preceding canonical module-decoder run.

3.5 GPU Implementation

Training is implemented in Burn [10]. Conditions, target splats, recurrent pools, generated adapters, and optimizer state remain on the GPU. CubeCL kernels provide tiled forward and explicit vector-Jacobian product operators for SPH perception and Target2D splatting; their adjoints have parity tests against the dense Burn objective. Per-sample adapter application retains the image batch dimension throughout both MLP matrices. CUDA and WGPU share the trainer and objective; only backend dispatch and kernels differ. Bevy inference materializes the adapter once and runs the optimized NPA hot path; headless figures use the same WGPU simulation and Gaussian-splatting renderer as the interactive viewer [1, 8].

4 Evaluation Protocol

4.1 Data and Splits

OmniSVG-1k. We use the first 1,000 cached MMSVG-Illustration training images. A deterministic identity split assigns every tenth item to holdout (offset zero), producing 900 train and 100 holdout identities. The fixed quality subset contains 16 holdout identities. No holdout image carries a teacher or direct adapter; the loader aborts if this invariant is violated.

Released NPA catalog. Ten official growing-2D targets and checkpoints are imported from the released NPA implementation [4].

Table 1: Evaluation hierarchy. “Train” rows cannot establish image generalization.

Controller	Condition	Split	Purpose
Released NPA	target-specific optimization	target	oracle
Direct table	sample ID	train	substrate ceiling
Shared base	none	holdout	unconditional control
Shuffled HyperNPA	wrong image	holdout	condition control
HyperNPA	unseen image	holdout	amortized model
Target splat	target points	holdout	renderer ceiling

The strict catalog experiment trains its conditioner on eight identities and holds out *rose* and *tropical fish*. These two official checkpoints provide the only same-target independent NPA oracle comparison in this study. The lizard checkpoint is reported as a renderer/reference sanity check, not a paired HyperNPA holdout.

4.2 Baselines and Controls

The *direct table* selects one canonical dense-delta row per training identity and bypasses DINO and the hypernetwork. It tests whether a shared base plus adapters can fit multiple targets under the same recurrent objective. It is not a generalized model and is never called an oracle. The *target splat* directly renders the extracted 4,096 target points and bounds losses caused by target preprocessing, particle count, and splat resolution.

4.3 Metrics

Numerical validation uses the differentiable Target2D renderer. For MSE e on unit-range values, PSNR = $-10 \log_{10} e$. We report:

- **composited RGB PSNR**, on $C + (1 - D)1$;
- **foreground RGB PSNR**, on straight RGB weighted only by target foreground occupancy;
- **density PSNR** after clamping D, D^* to $[0, 1]$;
- **density soft IoU**, $\sum \min(D, D^*) / \sum \max(D, D^*)$;
- **p10 and minimum PSNR**, exposing the low-quality tail;
- **condition gap**, correct minus shuffled-condition PSNR; and
- **adapter gain**, generated-controller minus base PSNR.

The quality checkpoint is selected by the minimum p10 over horizons of at least 256 steps, rather than training loss or mean PSNR. White-background composited PSNR can favor sparse images and cannot substitute for foreground, density, IoU, and visual evidence.

4.4 Common Rollout Contract

Unless stated otherwise, validation starts from a uniform circle of radius 0.2, uses seed 42, update probability 0.5, and horizons $T \in \{96, 256, 512, 1024\}$. OmniSVG numerical validation uses 2,048 particles; the target-splat ceiling uses 4,096 extracted points. Catalog numerical validation and every paper screenshot use 4,096 particles. Screenshots are raw 512^2 Bevy/WGPU captures with a black viewer background; reported PSNR comes from the alpha-aware differentiable renderer, not screenshot pixels.

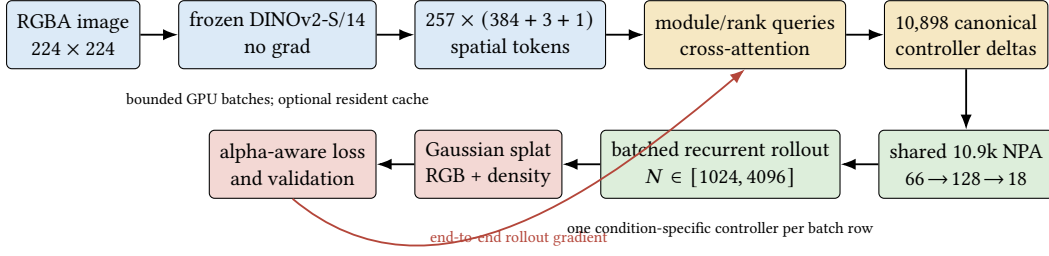


Figure 2: HyperNPA training and inference. The image encoder is frozen. The conditioner and, in the selected refinement, the shared NPA base receive gradients only through the rendered recurrent rollout objective. Evaluation materializes the generated controller once, then uses the same NPA inference path as released target-specific models.

Table 2: Strict same-target catalog comparison at 4,096 particles and 1,024 steps. Oracle checkpoints are released target-specific NPA models; HyperNPA never saw either target or adapter during training.

Target	Controller	Comp.	Fore.	Dens.	IoU
Rose	HyperNPA	9.63	8.85	7.78	.403
Rose	released oracle	27.93	27.57	26.89	.970
Fish	HyperNPA	11.65	10.31	8.46	.593
Fish	released oracle	27.45	26.65	26.61	.970
Lizard	released oracle	21.08	24.51	19.07	.881

Table 3: Selected OmniSVG-1k holdout result at 2,048 particles. Aggregate metrics pool pixel MSE before converting to PSNR; p10 is over 16 identities.

Metric at $T = 1024$	Result
Aggregate composited RGB PSNR	11.63 dB
Mean / p10 / minimum PSNR	12.70 / 9.32 / 8.30 dB
Aggregate foreground RGB PSNR	9.56 dB
Aggregate density PSNR	8.77 dB
Mean density soft IoU	0.536
Correct condition over shuffled	2.79 dB
Generated controller over shared base	3.26 dB
Target-point-splat p10 ceiling	28.08 dB
Gap to target-point-splat p10	18.77 dB

5 Results

5.1 Strict Released-Oracle Comparison

HyperNPA trails the paired oracle by 18.30 dB on rose and 15.80 dB on fish. Its two-target aggregate is 10.52 dB, foreground PSNR is 9.52 dB, density PSNR is 8.11 dB, and mean IoU is 0.498. The generated controller adds 2.33 dB over the shared base, but the correct-over-shuffled gap is only 0.09 dB. Figure 3 shows that the controller recovers broad hue and occupancy while missing target topology. The released controllers form the expected shapes and remain coherent at step 1,024.

5.2 Identity-Disjoint OmniSVG-1k

Table 3 proves that both condition and generated adapter paths affect held-out behavior. It also rejects particle extraction as the primary explanation: the same targets can be splatted above 28 dB p10. Yet all 16 examples miss the 26 dB gate and pairwise generated adapters remain similar (mean cosine 0.992), consistent with an overly unconditional shared attractor.

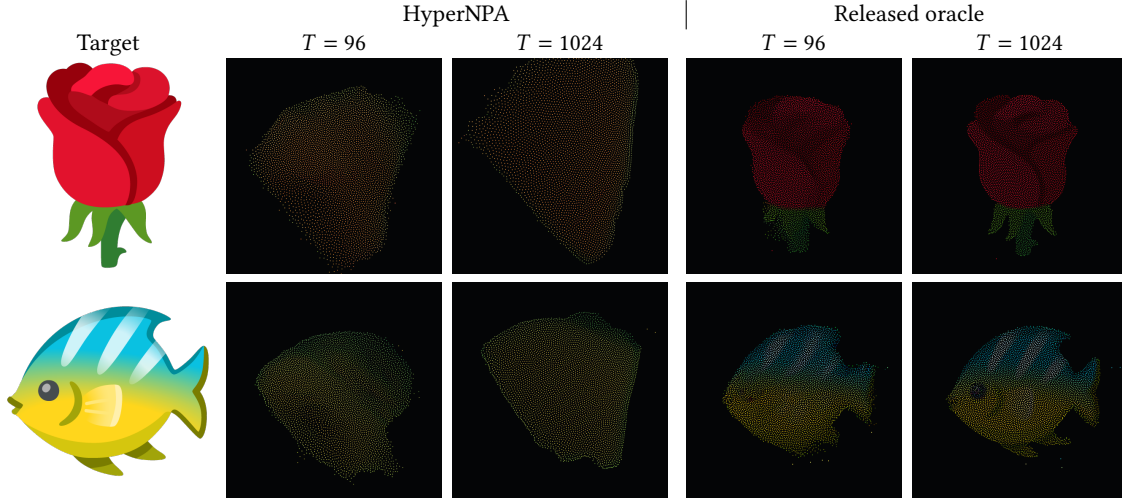


Figure 3: Held-out catalog rollouts under a common 4,096-particle contract. The comparison holds target, seed, asynchronous update probability, renderer, and horizon fixed.

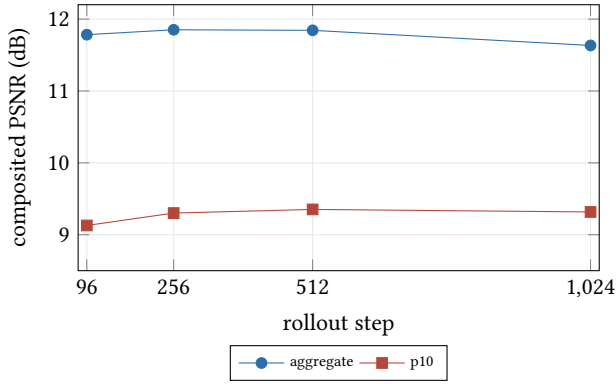


Figure 4: The selected 1k checkpoint is dynamically stable but stably low-quality. Peak-to-final p10 loss is only 0.035 dB; more inference steps do not close the quality gap.

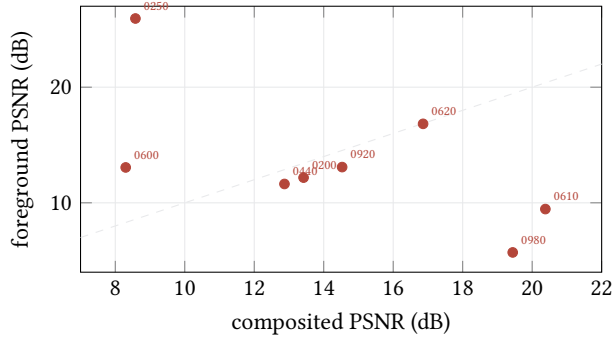


Figure 5: Whole-image composited PSNR and alpha-weighted foreground PSNR can disagree sharply. IDs correspond to Figure 6.

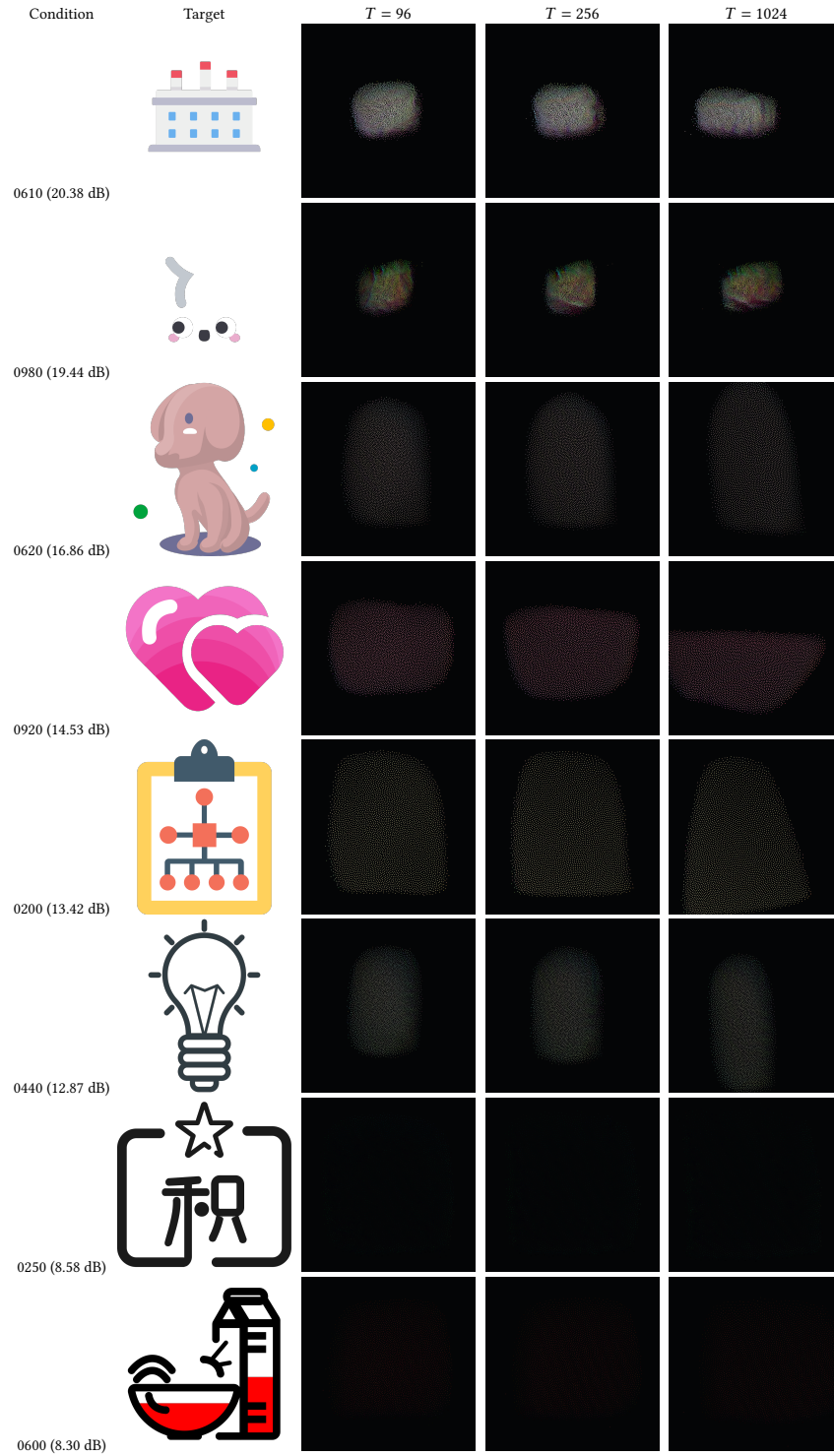


Figure 6: Eight identity-disjoint OmniSVG holdouts. Every rollout uses 4,096 particles; parenthesized metrics are the corresponding 2,048-particle Target2D composited PSNR at $T = 1024$. Even the highest-scoring building is a coarse mass, the hearts become a wedge, and sparse line art largely disappears. The trajectories are stable in time but not semantically faithful.

5.3 Shared-Substrate Capacity Control

The direct adapter table removes image encoding and generation. Sixteen training identities jointly optimize the shared base and one canonical delta per identity under progressively denser rollouts. The campaign matches upstream compute exposure by counting a trajectory at N particles as $N/4096$ of a 4,096-particle trajectory. The 512-, 1,024-, and 2,048-particle stages total 250,000 equivalent trajectories per identity, or 104.17% of the released recipe’s 240,000. All rows are evaluated at 4,096 particles.

Increased particle density and exposure produced large gains in this legacy control. However, its canonical adapter trained an image-specific output bias, whereas the released Growing NPA has no final-layer bias. The entire table is therefore retained only as historical optimization evidence and cannot bound the zero-bias conditioned architecture. A bounded corrected control used an upstream-seeded shared rule, zero-output-bias rank-82 controllers, 4,096 particles, eight trajectories per identity, and rollout lengths sampled from 32–95. After 2,000 joint updates (16,000 trajectories per identity, 6.67% of the released exposure), it reached 11.63 dB aggregate and 10.52 dB p10 at 1,024 steps; its minimum multi-horizon p10 was 10.43 dB. Correct controllers outperformed shuffled identities by 3.77 dB, confirming that the substrate was active, but the result is far below parity and is not a converged capacity ceiling. A full-exposure parity-validated control remains necessary before attributing the gap to amortization alone.

5.4 Metric Failure Modes

The 16 held-out examples reveal why one score is insufficient. Condition 0610 has the highest composited PSNR (20.38 dB) but only 9.46 dB foreground PSNR; its building becomes a colored blob. Condition 0980 obtains 19.44 dB composited PSNR yet only 5.71 dB foreground PSNR and 0.152 IoU. Conversely, condition 0250 has 25.94 dB foreground color PSNR but 8.58 dB composited PSNR because it predicts approximately the right color in the wrong occupancy. These are not numerical edge cases: Figure 6 shows the corresponding 4,096-particle states.

5.5 Training Throughput

The current v3 CUDA profile uses 64 independent conditions, 1,024 particles, 96 full-BPTT steps, release mode, and an NVIDIA RTX PRO 6000 Blackwell (96 GiB, 600 W). Validation and checkpoint I/O are disabled. Exact optimizer exposure is 1.887 billion particle-steps over 300 updates.

This is an 8.5× increase over the selected WGPU quality refinement’s 2.08 Mparticle-steps/s median, but hardware, backend, batch size, and decoder version all change, so it is not a controlled kernel-only speedup. The profile does establish that the current trainer can sustain tens of millions of particle updates per second without host perception fallback. Quality-scale 4,096-particle backward remains the expensive regime.

6 Discussion

6.1 What Has Been Demonstrated

The experiments support three positive conclusions. First, per-sample conditioned controllers are correctly batched and applied:

Table 4: Historical non-amortized 16-identity diagnostic. These runs predate the zero-output-bias parity contract and are not valid oracle ceilings. Throughput is median optimizer throughput.

Training continuation	Agg.	p10	Gap	Mparticle-steps/s
512p, 240k traj./ID	13.06	10.82	4.85	13.26
+1024p, 240k traj./ID	15.87	13.24	7.79	10.21
+2048p, 320k traj./ID	20.71	18.05	12.60	6.90
+4096p density cycle	20.71	18.05	12.60	4.11
frozen base, table refine	23.58	19.30	15.46	4.03

Table 5: Current module-token v3 systems benchmark. It demonstrates trainer throughput, not the quality of the selected v2 checkpoint.

Metric	Result
Measured end-to-end optimizer throughput	17.66 Mparticle-steps/s
Median report-window throughput	18.36 Mparticle-steps/s
Minimum / maximum report window	15.93 / 18.76 Mparticle-steps/s
Training wall time	109.1 s
Condition batch / particle count / steps	64 / 1024 / 96
Perception custom-op device hits	28,800 forward
Perception VJP device hits	28,500 backward
Host perception fallbacks	0

correct images outperform shuffled images by 2.79 dB on OmniSVG, and generated adapters outperform the shared base by 3.26 dB. Second, the target representation and 4,096-particle renderer can express high-quality outputs: direct target splats reach 28.08 dB p10 and released rose/fish controllers reach about 27.5 dB. Third, conditional control, runtime materialization, and GPU training are alive. The historical 23.58 dB table result is not counted as parity evidence because it used a per-image output bias unavailable to the released NPA.

6.2 What Has Not Been Demonstrated

The present model does not achieve oracle-quality or broad high-fidelity generalization. The primary 1k numerical result covers only 16 of 100 held-out identities, one deterministic split, and one rollout seed. The catalog oracle comparison covers two identities. We report no confidence intervals, no damage-regeneration test for generated controllers, and no independently overfit NPA oracle bank for the OmniSVG holdouts. The best visual sample is not faithful despite a 20.38 dB composited score.

The studied decoder is deterministic, so this work makes no rectified-flow quality claim. DINO is frozen and was not pretrained for alpha-aware vector graphics. Static controller deltas must encode the entire target layout; particles do not directly sample a condition field during rollout. Finally, the Burn objective adds white-composited RGB and strong boundary terms to the released density/color objective. These choices are explicit, but strict upstream training parity should be established before attributing every gap to hypernetwork architecture.

6.3 Evidence-Based Next Steps

The control ladder suggests the following order:

- (1) **Establish an independent Burn oracle.** Match released target extraction, seed pool, $N = 4096$, $T \in [32, 96]$, density-gated color loss, damage, optimizer, and 30k iterations; compare one-step values, gradients, updates, and long rollouts to the released implementation.
- (2) **Raise the adapter substrate tail.** Continue multi-identity direct-adapter training under the zero-output-bias contract with a frozen, parity-validated trunk and select on 4,096-particle multi-horizon p10. No valid 26 dB amortization ceiling has yet been established.
- (3) **Distill function, then fine-tune end to end.** Warm-start the v3 token generator from canonical dense deltas or probe-rollout behavior, then optimize downstream rollout loss with the trunk frozen before a small joint-learning phase.
- (4) **Add spatial runtime memory.** Let particles bilinearly sample a compact condition feature grid in addition to static controller modulation. This removes pressure to encode exact image layout entirely in 10.9k weights.
- (5) **Only then test stochastic flow.** Train an adapter velocity model with noisy canonical deltas and time conditioning, then validate sampled controllers through the same oracle-relative rollout suite.

7 Conclusion

HyperNPA converts a condition image into a runnable NPA controller in one feed-forward pass and executes that controller through a common optimized particle runtime. The maintained implementation preserves spatial DINO tokens, generates identifiable module-structured deltas, trains through the actual recurrent image objective, and exposes strict conditional and tail metrics. At present, it is an auditable baseline rather than a solved generative model: held-out quality is 11.63 dB aggregate and 9.32 dB p10, while released target-specific controllers exceed 27 dB on paired targets. The direct-adapter control shows a plausible route upward, but also establishes a hard ordering: first validate and optimize the shared recurrent substrate, then amortize it, and only then add a stochastic adapter generator.

Reproducibility Statement

The repository records TOML experiment definitions, binary checkpoints, JSON reports, deterministic split rules, seeds, and headless renderer reports. The paper’s central evidence comes from:

- OmniSVG-1k: artifacts/hyper2d_omnisvg_1k_dino_canonical_quality_refine/report.json;
- catalog holdout: artifacts/hyper2d_growing_catalog_holdout2_quality_eval_p4096_s1024/report.json;
- historical non-parity table diagnostic: artifacts/hyper2d_omnisvg_16_p4096_frozen_table_lr2e6/report.json; and
- throughput: artifacts/throughput_v3_p1024_s96_b64_sources64_cuda_release_fixed96_retained_vjp/report.json.

Table 6: The eight OmniSVG holdouts visualized in Figure 6. Metrics use 2,048 particles and $T = 1024$; figures use 4,096 particles.

ID	Comp.	Fore.	Density	Soft IoU
0610	20.38	9.46	12.46	.673
0980	19.44	5.71	13.40	.152
0620	16.86	16.83	10.21	.607
0920	14.53	13.09	8.92	.585
0200	13.42	12.18	10.87	.782
0440	12.87	11.63	10.84	.273
0250	8.58	25.94	7.73	.269
0600	8.30	13.06	8.25	.401

All paper rollouts were regenerated with the headless export command, 4,096 particles, seed 42, update probability 0.5, and captures at steps 0, 32, 96, 256, 512, and 1,024. Appendix A lists the eight visualized holdout metrics.

A Per-Sample Held-Out Metrics

B Artifact and Version Boundaries

The best 1k quality checkpoint serializes architecture module_token_cross_attention_lora_v2. The source tree’s maintained default is module_token_multihead_cross_attention_lora_v3. We deliberately report the architecture stored in each artifact instead of relabeling old weights as the latest code. Likewise, historical directory names containing rectified_flow do not establish a flow objective. The report field generator_sample_steps=1, zero source noise, and deterministic artifact inference define the evaluated behavior.

The imported catalog metrics evaluate released target-specific checkpoints in the Burn quality harness; they do not claim that the current Burn trainer can reproduce those checkpoints from scratch. The direct adapter table uses the same NPA runtime and loss as HyperNPA, but evaluates training identities and must not be compared as held-out generalization.

C Maintained Conditional Row-Flow Path

The current trainer preserves all 257 DINOv2 tokens and appends the aligned 14×14 RGBA patch payload to each of the 256 patch tokens. Deterministic seeded source rows for the two NPA affine layers are transported by a timestep-conditioned self/cross-attention velocity network. An eight-step Heun solve produces 10,898 identifiable controller residuals. End-to-end image and rollout loss updates both the generated residual and shared NPA trunk. Detached TBPTT accumulates controller-endpoint VJPs on device and contracts them through the flow graph once per optimizer step.

This architecture is stored as version 5 inside the versioned BPK artifact container and remains load-compatible with legacy version-4 row-flow artifacts. Its bounded four-condition control shows active conditioning, but its p10 quality remains far below the 26 dB target. Consequently, broad generalized row-flow quality is an open result, not a paper conclusion.

References

- [1] Bevy Foundation. 2026. Bevy Engine. Software. <https://bevyengine.org/>
- [2] David Ha, Andrew Dai, and Quoc V. Le. 2016. HyperNetworks. *arXiv preprint arXiv:1609.09106* (2016). <https://arxiv.org/abs/1609.09106>

- [3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685* (2021). <https://arxiv.org/abs/2106.09685>
- [4] Hyunsoo Kim and collaborators. 2026. Neural Particle Automata: Official Implementation and Models. GitHub repository. <https://github.com/TheDevilWillBeBee/NPA>
- [5] Hyunsoo Kim, Ehsan Pajouheshgar, Sabine Süsstrunk, Wenzel Jakob, and Jinah Park. 2026. Neural Particle Automata: Learning Self-Organizing Particle Dynamics. *arXiv preprint arXiv:2601.16096* (2026). <https://arxiv.org/abs/2601.16096>
- [6] Alexander Mordvintsev, Ettore Randazzo, and Craig Fouts. 2022. Growing Isotropic Neural Cellular Automata. *arXiv preprint arXiv:2205.01681* (2022). <https://arxiv.org/abs/2205.01681>
- [7] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. 2020. Growing Neural Cellular Automata. *Distill* (2020). doi:10.23915/distill.00023
- [8] mosure. 2026. bevy_gaussian_splatting. GitHub repository. https://github.com/mosure/bevy_gaussian_splatting
- [9] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. 2024. DINOv2: Learning Robust Visual Features without Supervision. *Transactions on Machine Learning Research* (2024). <https://arxiv.org/abs/2304.07193>
- [10] Tracel AI. 2026. Burn: A Flexible Deep Learning Framework for Rust. Software. <https://burn.dev/>
- [11] Yiyang Yang, Wei Cheng, Sijin Chen, Xianfang Zeng, Fukun Yin, Jiaxu Zhang, Liao Wang, Gang Yu, Xingjun Ma, and Yu-Gang Jiang. 2025. OmniSVG: A Unified Scalable Vector Graphics Generation Model. *arXiv preprint arXiv:2504.06263* (2025). <https://arxiv.org/abs/2504.06263>